

ESTUDIO — Carpeta repositories/

Material de estudio para el final del TFC Bajoneá. Basado en el código real de `backend/src/main/java/com/bajonea/backend/repositories/` — **26 repositories**, uno por entidad.

Qué es un Repository

Un **repository** es la capa que habla con la base de datos. Es la única parte del proyecto que sabe que existe MySQL — el resto trabaja con objetos Java.

Lo llamativo de esta carpeta: casi no tiene código. Son todas **interfaces**, sin ninguna implementación escrita a mano.

```
public interface CategoriaRepository extends JpaRepository<Categoria, Integer> {
    boolean existsByNombre(String nombre);
    long countByActivoTrue();
}
```

Eso es un archivo entero. **Spring Data JPA genera la implementación en tiempo de ejecución**, y vos podés inyectar `CategoriaRepository` en un service y usarlo como si tuviera código adentro.

Qué te da JpaRepository gratis

Al extender `JpaRepository<Entidad, TipoDeId>` heredás un montón de métodos sin escribir nada:

Método heredado	Qué hace
<code>save(entidad)</code>	Inserta si es nueva, actualiza si ya existe.
<code>findById(id)</code>	Busca por clave primaria. Devuelve <code>Optional</code> .
<code>findAll()</code>	Trae todas las filas.
<code>deleteById(id)</code>	Borra por id.
<code>existsById(id)</code>	Devuelve <code>true</code> / <code>false</code> .
<code>count()</code>	Cuenta las filas.
<code>saveAll(lista)</code>	Guarda varias de una.

Los dos parámetros del genérico son la **entidad** y el **tipo de su id**. Fijate que casi todos usan `Integer` , pero `ProvinciaRepository` y `LocalidadRepository` usan `String` (sus ids son códigos oficiales de la API Georef), y `ProductoTagRepository` usa `ProductoTagId` (clave compuesta).

Las "query methods" — la magia de Spring Data

Esta es la parte que más suele sorprender en una mesa. **Spring Data lee el nombre del método y genera la consulta SQL a partir de él.**

```
List<Producto> findByComercioId(Integer comercioId);
```

Spring lo lee palabra por palabra: `find + By + ComercioId` → *"buscá productos donde el comercio tenga este id"*, y genera:

```
SELECT * FROM producto WHERE comercio_id = ?
```

El vocabulario que se usa en el proyecto

Palabra clave	Qué hace	Ejemplo real del proyecto
findBy	Buscar	findByEmail(String email)
existsBy	¿Existe? Devuelve boolean	existsByDni(String dni)
countBy	Contar. Devuelve long	countByCategoriaId(Integer id)
deleteBy	Borrar	deleteByProductoId(Integer id)
And	Combinar dos condiciones	findByComercioIdAndEstado(...)
Not	Distinto de	findByComercioIdAndEstadoNot(...)
True / False	Booleano fijo	countByActivoTrue()
IsNull	Campo nulo	findByComercioIdAndFechaBajaIsNull(...)
Between	Entre dos valores	findByComercioIdAndFechaCreacionBetween(...)
OrderBy...Asc/Desc	Ordenar	findByProductoIdOrderByOrdenAsc(...)
Top / First	Traer solo el primero	findTopByComercioIdOrderByFechaHoraDesc(...)
_ (guion bajo)	Navegar a una relación	findByComercio_EstadoAndEstadoNot(...)

La ventaja: no escribís SQL, y si te equivocás en el nombre de un campo **la aplicación no arranca** — Spring valida los nombres al iniciar. Un error de tipeo en un SQL escrito a mano recién explotaría en producción.

La desventaja: los nombres se vuelven larguísimos. `findFirstByUsuarioIdAndTipoAndEstadoOrderByFechaCreacionDesc` es un método real del proyecto. Cuando llegás a eso, conviene pasar a `@Query` .

Por qué Optional<T> en vez de devolver null

Muchos métodos devuelven `Optional<Usuario>` en lugar de `Usuario` . Es una forma de decir "**esto puede venir vacío**" de manera explícita, obligando a quien lo usa a manejar ese caso:

```
Usuario usuario = usuarioRepository.findByEmail(email)
    .orElseThrow(() -> new RecursoNoEncontradoException("Usuario no encontrado"));
```

Sin `Optional` , olvidarse de chequear el null es un `NullPointerException` esperando a pasar.

La regla del proyecto: los repositories son "tontos"

Está escrito en CLAUDE.md §3: "*repositories/ queda tonto (solo queries derivadas o @Query); la lógica de negocio vive en services/*".

Traducido: un **repository** solo sabe traer y guardar datos. **Nunca decide nada**. Si hay que comparar, calcular o validar, eso pasa en el service.

LOS 26 REPOSITORIES

Grupo A: los que no agregan nada (5)

Repository	Entidad	Por qué está vacío
AdministradorRepository	Administrador	Con <code>findById</code> alcanza — se busca por el id del usuario logueado.
ClienteRepository	Cliente	Ídem.
DuenoRepository	Dueno	Ídem.
PersonaRepository	Persona	Es el nodo intermedio, se accede por id.
ProvinciaRepository	Provincia	Solo se necesita <code>findAll()</code> (las 24 provincias).

Si te preguntan "**¿por qué existen si están vacíos?**": porque necesitás el objeto repository para llamar a los métodos heredados (`save` , `findById` , `findAll`). Sin la interfaz, no hay nada que inyectar.

Grupo B: identidad y seguridad

UsuarioRepository — el más interesante de todos

Método	Qué consulta en criollo
findByEmail(String email)	Trae el usuario con ese email. Se usa en el login y en la recuperación de contraseña.
existsByEmail(String email)	¿Ya hay alguien con ese email? Se usa al registrar.
findByEmailConBloqueo(String email)	Igual que el primero, pero con bloqueo pesimista .
findByIdConBloqueo(Integer id)	Igual, pero buscando por id.

Los dos métodos con @Lock — el tema estrella de esta carpeta

Son los únicos dos del proyecto que usan `@Query` explícito, y llevan una anotación especial:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT u FROM Usuario u WHERE u.email = :email")
Optional<Usuario> findByEmailConBloqueo(String email);
```

Qué hace `PESSIMISTIC_WRITE` : genera un `SELECT ... FOR UPDATE` en SQL. Eso **bloquea esa fila en la base** hasta que termine la transacción. Cualquier otra transacción que quiera tocar el mismo usuario **queda esperando**.

El problema que resuelve, con un ejemplo concreto

Imaginate un doble submit de login (el usuario hace doble clic en "Entrar", con la contraseña mal):

Sin bloqueo:

```
Request A: lee usuario, intentosFallidos = 2
Request B: lee usuario, intentosFallidos = 2      (al mismo tiempo)
Request A: escribe 3
Request B: escribe 3                             (pisa lo de A)
Resultado: 3 en vez de 4. El bloqueo nunca llega.
```

Eso se llama **lost update** — una actualización se pierde. Con contadores de seguridad es grave: un atacante con requests paralelos podría probar contraseñas casi sin límite.

Con bloqueo:

```
Request A: lee CON LOCK, intentosFallidos = 2
Request B: quiere leer... ESPERA
Request A: escribe 3, termina la transacción
Request B: ahora lee 3, escribe 4 
```

Pesimista vs optimista — buena pregunta de mesa

	Pesimista (lo que se usa acá)	Optimista
Cómo funciona	Bloquea la fila desde el principio	Deja pasar a todos y detecta el choque al guardar (con una columna <code>version</code>)
Cuándo conviene	Cuando el conflicto es probable	Cuando el conflicto es raro
Costo	Otros esperan	Reintentar cuando choca

Se eligió pesimista porque el login concurrente sobre el mismo usuario es un escenario realista, y porque acá se está protegiendo un **contador de seguridad**: es preferible que alguien espere unos milisegundos antes que perder un intento fallido.

El detalle de criterio que vale la pena mencionar: el comentario del propio código aclara "*usar únicamente donde la transacción va a mutar el usuario leído; para lecturas simples usar `findByEmail` sin lock*". **No se bloquea de más**. Un `SELECT FOR UPDATE` en cada lectura mataría el rendimiento sin ganar nada. Hay dos versiones del mismo método justamente para poder elegir.

(El análisis completo está en `docs/CONCURRENCIA-Y-TRANSACCIONES.md` , sección 1.)

SesionRepository

Método	Qué consulta
findByUsuarioIdAndActivaTrue(Integer usuarioId)	La sesión abierta de ese usuario, si tiene alguna.

Devuelve `optional` , o sea que espera como mucho una. Eso refleja una regla de negocio real: **sesión única por cuenta**. Cuando alguien se loguea de nuevo, la sesión anterior se cierra.

Se usa para: cerrar la sesión previa en un login nuevo, y para el logout.

(El `findById` que usa `JwtAuthenticationFilter` en cada request es el heredado, no está declarado acá.)

TokenRepository

Método	Qué consulta
findByTokenAndEstado(String token, EstadoToken estado)	El token con ese valor y en ese estado.
findByUsuarioIdAndTipoAndEstado(...)	Todos los tokens de un usuario, de cierto tipo y estado.
findFirstByUsuarioIdAndTipoAndEstadoOrderByFechaCreacionDesc(...)	El más reciente de esos.

Por qué el primero busca por token y estado juntos: para que solo sirva un token `PENDIENTE` . Si buscara solo por el valor, encontraría uno ya `UTILIZADO` y habría que chequearlo aparte. Poniendo el estado en la consulta, un token usado directamente no aparece. Un solo uso garantizado.

El segundo se usa para invalidar los tokens viejos: cuando pedís un código nuevo, los anteriores se marcan como usados. Si no, tendrías varios códigos válidos a la vez.

El tercero es solo para los tests. Lo usa `TestSupportService` (perfil `test`) para agarrar el token más reciente sin tener que leer un mail de verdad.

PersonaFisicaRepository y PersonaJuridicaRepository

Repository	Método	Qué consulta
PersonaFisicaRepository	existsByDni(String dni)	¿Ya hay alguien con ese DNI?
PersonaJuridicaRepository	existsByCuit(String cuit)	¿Ya hay una empresa con ese CUIT?

Ambos se usan en el registro. Y ambos son la primera línea de defensa, no la última: la garantía real es el `UNIQUE` de la base, porque entre el `existsBy` y el `save` puede meterse otro request (ver ESTUDIO-EXCEPTIONS.md, el handler de `DataIntegrityViolationException`).

Por qué se hace igual el `existsBy` : para poder devolver un mensaje claro ("ese DNI ya está registrado") en el 99,9% de los casos, en vez del mensaje genérico del `UNIQUE` .

Grupo C: comercio y catálogo

ComercioRepository

Método	Qué consulta
findByEstado(EstadoComercio estado)	Todos los comercios en ese estado.
findByDuenoId(Integer duenoId)	El comercio de ese dueño.

El primero es el que sostiene todo el catálogo público: `findByEstado(APROBADO)` . Un comercio `PENDIENTE` o `RECHAZADO` no aparece. También se usa con `PENDIENTE` para el panel del admin.

El segundo devuelve `Optional` , o sea uno solo, aunque el modelo dice que un dueño puede tener varios comercios (`Dueno 1 → N Comercio`).

Vale la pena decirlo así en la mesa: "el modelo de datos ya soporta que un dueño tenga varios comercios, pero la implementación actual del service todavía trabaja con uno solo por dueño. Cuando se implemente el multi-comercio, este método pasaría a devolver una lista." A confirmar en el service cuál es el comportamiento exacto si el tribunal profundiza.

ProductoRepository

Método	Qué consulta
findByComercioIdAndEstadoNot(Integer comercioId, EstadoProducto estado)	Productos de un comercio, excluyendo un estado. Se usa con <code>DESCONTINUADO</code> para el catálogo público.
findByComercio_EstadoAndEstadoNot(EstadoComercio, EstadoProducto)	Productos de todos los comercios aprobados , sin los descontinuados. Para la búsqueda global.
findByComercioIdAndCategoriaId(...)	Productos de un comercio filtrados por categoría.
findByComercioId(Integer comercioId)	Todos los productos del comercio, sin filtro de estado.
countByCategoriaId(Integer categoriaId)	Cuántos productos usan esa categoría.

El contraste entre el primero y el cuarto — buen detalle para contar

Son casi iguales, pero se usan en lugares opuestos:

Método	Quién lo usa	Qué ve
findByComercioIdAndEstadoNot(id, DECONTINUADO)	El público	Todo menos lo descontinuado
findByComercioId(id)	El propio comercio	Todo , incluidos agotados y descontinuados

Tiene lógica: el cliente no tiene por qué ver productos que ya no se venden; el comercio sí necesita verlos para poder reactivarlos o editarlos.

El guion bajo en findByComercio_EstadoAndEstadoNot

Ese `_` es sintaxis de Spring Data para navegar a una relación:

- Comercio_Estado → "el campo estado **del comercio relacionado**"
- EstadoNot → "el campo estado **del producto**"

Sin el guion bajo, Spring no sabría a cuál de los dos estado te referís. Genera un JOIN con la tabla comercio.

countByCategoriaId

Es el que llena el cantidadProductos de CategoriaResponseDTO, para que en el panel de admin se lea "Pizzas — 12 productos asociados". Un COUNT(*) en SQL es mucho más barato que traer las 12 filas para contarlas en Java.

ProductoTagRepository

Es el único con clave compuesta: JpaRepository<ProductoTag, ProductoTagId>.

Método	Qué consulta
findByProductoId(Integer productoId)	Los tags de un producto.
findByTagId(Integer tagId)	Los productos que tienen ese tag.
deleteByProductoId(Integer productoId)	Borra todas las asociaciones de un producto.
countByTagId(Integer tagId)	Cuántos productos usan ese tag.

El par deleteByProductoId + findByProductoId implementa la estrategia de edición de tags: cuando editás un producto, en vez de calcular qué tags agregar y cuáles sacar (que sería propenso a errores), se borra todo y se vuelve a crear. Más simple y sin casos raros. Al ser pocos tags por producto (máximo 5), el costo es despreciable.

CategoriaRepository y TagRepository

Ambos idénticos:

Método	Qué consulta
existsByNombre(String nombre)	¿Ya hay una con ese nombre? Evita duplicados.
countByActivoTrue()	Cuántas están activas (no dadas de baja).

countByActivoTrue alimenta las métricas del dashboard del admin.

ImagenProductoRepository

Método	Qué consulta
findByProductoIdOrderByOrdenAsc(Integer productoId)	Las imágenes de un producto, ya ordenadas .
countByProductoId(Integer productoId)	Cuántas tiene.

El orderByOrdenAsc hace que ordene la base, no Java. Es lo correcto: una base de datos ordena mucho más rápido, sobre todo si hay un índice.

El countByProductoId es el que hace cumplir el límite de 5 imágenes. El service lo llama antes de agregar: si ya hay 5, tira ConflictDeNegocioException → 409.

RedSocialRepository

Método	Qué consulta
findByComercioIdAndFechaBajaIsNull(Integer comercioId)	Las redes activas de un comercio.
countByComercioIdAndFechaBajaIsNull(Integer comercioId)	Cuántas activas tiene.
findByComercioIdAndTipo(Integer comercioId, TipoRedSocial tipo)	Su red de un tipo puntual.

El FechaBajaIsNull es la implementación de la baja lógica. Una red dada de baja tiene fecha en fechaBaja, así que preguntar por IS NULL trae solo las vigentes. La fila nunca se borra.

El tercero sirve para evitar duplicados: no tiene sentido que un comercio cargue dos Instagram.

El segundo hace cumplir el límite de 5 redes por comercio.

HorarioRepository

Método	Qué consulta
findByComercioId(Integer comercioId)	Todas las franjas horarias del comercio.

Grupo D: carrito y pedido

CarritoRepository

Método	Qué consulta
<code>findByClienteId(Integer clienteId)</code>	El carrito de ese cliente.

Devuelve `optional` , no `List` , y eso es la regla de negocio: **un solo carrito por cliente**. Reforzada además por el `unique = true` en la columna `cliente_id` .

ItemCarritoRepository

Método	Qué consulta
<code>findByCarritoId(Integer carritoId)</code>	Los ítems de un carrito.
<code>deleteByCarritoId(Integer carritoId)</code>	Los borra todos (vaciar el carrito).
<code>findByProductoId(Integer productoId)</code>	Todos los ítems, de cualquier carrito, que tengan ese producto.

El tercero es el más interesante

LO USA `ProductoService.limpiarCarritosActivos` . La historia es esta:

Quando un producto pasa a **AGOTADO** o **DESCONTINUADO** , hay clientes que ya lo tienen en el carrito. No podés dejarlo ahí, porque cuando confirmen el pedido va a fallar.

Entonces el service:

- Busca **todos** los ítems de **todos** los carritos con ese producto.
- Notifica a cada cliente afectado** (`PRODUCTO_REMOVIDO_CARRITO`).
- Recién ahí borra los ítems.

El **orden importa**: notificar antes de borrar. Si borraras primero, ya no sabrías a quién avisar.

Detalle sobre el nombre `limpiarCarritosActivos` : en el modelo actual no hay concepto de "carrito activo vs inactivo" — hay un solo carrito por cliente, y son todos. El nombre viene de una versión anterior del diseño.

Y acá se encontró un **bug real**: cuando el carrito quedaba **vacío** por esta limpieza, no se le reseteaba el campo `comercio` . Resultado: el cliente quedaba "atado" a un comercio pero con el carrito vacío, y no podía agregar productos de otro. Se corrigió poniendo `comercio = null` cuando el carrito queda sin ítems.

PedidoRepository

Método	Qué consulta
<code>findByClienteId(Integer clienteId)</code>	El historial de pedidos de un cliente.
<code>findByComercioId(Integer comercioId)</code>	Los pedidos que recibió un comercio.
<code>findByComercioIdAndEstado(Integer comercioId, EstadoPedido estado)</code>	Los pedidos de un comercio en cierto estado.
<code>findByComercioIdAndFechaCreacionBetween(Integer comercioId, LocalDateTime inicio, LocalDateTime fin)</code>	Los pedidos de un comercio entre dos fechas .

Los dos primeros son los que hacen segura la consulta de pedidos. Como el `clienteId` sale del JWT y no de la URL, **es imposible que te devuelvan pedidos ajenos**. Por eso el proyecto no tiene un `GET /pedidos/{id}` : el frontend trae la lista (ya filtrada) y busca adentro.

El cuarto alimenta el **resumen del dashboard**: se le pasa el inicio y el fin del día de hoy, y devuelve los pedidos de la jornada. Después el service calcula el total facturado y los conteos.

DetallePedidoRepository

Método	Qué consulta
<code>findByPedidoId(Integer pedidoId)</code>	Las líneas de un pedido.

Grupo E: soporte

NotificacionRepository

Método	Qué consulta
findByUsuarioIdOrderByFechaCreacionDesc(Integer usuarioId)	Sus notificaciones, de la más nueva a la más vieja .
findByUsuarioIdAndLeidaFalse(Integer usuarioId)	Solo las no leídas.
countByUsuarioIdAndLeidaFalse(Integer usuarioId)	Cuántas no leídas.

El tercero es el más llamado de todo el proyecto, porque es el del polling: el frontend lo consulta **cada 15 segundos** para el numerito de la campanita.

Por eso devuelve **solo un número**, no una lista. Un `COUNT(*)` es muchísimo más barato que traer todas las filas y contarlas en Java, y por la red viaja un entero en vez de un array de objetos. Es una optimización concreta y justificada por el patrón de uso.

DireccionRepository

Método	Qué consulta
findByComercioId(Integer comercioId)	La dirección de un comercio.
findByClienteId(Integer clienteId)	La dirección de un cliente.

Los dos devuelven `Optional`, o sea una sola.

Nota sobre la cardinalidad: el modelo permite que un cliente tenga **muchas** direcciones (`@ManyToOne` en `Direccion.cliente`), pero el método devuelve una sola, porque la implementación actual trabaja con una dirección por cliente. *(La gestión de múltiples direcciones quedó explícitamente fuera del alcance implementado.)*

Para qué se usa `findByClienteId` : para que el checkout de delivery sepa cuál es el `direccionId` real que tiene que mandar en el `PedidoRequestDTO` .

HistorialEstadoComercioRepository

Método	Qué consulta
findTopByComercioIdOrderByFechaHoraDesc(Integer comercioId)	La última transición de estado de ese comercio.

Cómo se lee el nombre: `findTop` = traeme solo el primero, `ByComercioId` = de ese comercio, `OrderByFechaHoraDesc` = ordenado de más nuevo a más viejo. O sea: **el cambio de estado más reciente**.

Para qué sirve: para saber **por qué** se rechazó un comercio. El modelo de datos eliminó la columna `Comercio.motivo_rechazo` a favor de esta tabla de historial, así que **el motivo solo vive acá**. Cuando el comercio rechazado entra a la app, este método trae su última transición y de ahí sale el motivo que se le muestra.

En SQL genera un `LIMIT 1` , o sea que la base trae una sola fila. Mucho mejor que traer todo el historial y quedarse con el primero en Java.

LocalidadRepository

Método	Qué consulta
findByProvinciaId(String provinciaId)	Las localidades de esa provincia.

Es el que alimenta el **selector dependiente**: elegís provincia y se cargan sus localidades. Sin este filtro habría que traer las ~4038 de una, lo cual sería absurdo tanto para la base como para el navegador.

El `id` es `String` porque son códigos oficiales de la API Georef, no números correlativos.

Tabla resumen de los 26 repositories

Repository	Entidad	Tipo del id	Métodos custom
UsuarioRepository	Usuario	Integer	4 (2 con @Lock)
SesionRepository	Sesion	Integer	1
TokenRepository	Token	Integer	3
PersonaRepository	Persona	Integer	0
PersonaFisicaRepository	PersonaFisica	Integer	1
PersonaJuridicaRepository	PersonaJuridica	Integer	1
ClienteRepository	Cliente	Integer	0
AdministradorRepository	Administrador	Integer	0
DuenoRepository	Dueno	Integer	0
ComercioRepository	Comercio	Integer	2
ProductoRepository	Producto	Integer	5
ProductoTagRepository	ProductoTag	ProductoTagId	4
CategoriaRepository	Categoria	Integer	2
TagRepository	Tag	Integer	2
ImagenProductoRepository	ImagenProducto	Integer	2
RedSocialRepository	RedSocial	Integer	3
HorarioRepository	Horario	Integer	1
CarritoRepository	Carrito	Integer	1
ItemCarritoRepository	ItemCarrito	Integer	3
PedidoRepository	Pedido	Integer	4
DetallePedidoRepository	DetallePedido	Integer	1
NotificacionRepository	Notificacion	Integer	3
DireccionRepository	Direccion	Integer	2
HistorialEstadoComercioRepository	HistorialEstadoComercio	Integer	1
ProvinciaRepository	Provincia	String	0
LocalidadRepository	Localidad	String	1

Preguntas típicas de mesa

"¿Qué es un repository?" La capa que habla con la base de datos. En mi proyecto son interfaces que extienden JpaRepository ; Spring Data genera la implementación sola en tiempo de ejecución.

"¿Cómo hacés una consulta sin escribir SQL?" Con "query methods": Spring lee el nombre del método (findByIdByComercioIdAndEstadoNot) y genera la consulta. Si el nombre menciona un campo que no existe, la aplicación no arranca — el error se detecta al iniciar, no en producción.

"¿Y si necesitás algo que el nombre no puede expresar?" Uso @Query con JPQL. En mi proyecto lo uso en dos métodos de UsuarioRepository , los que necesitan un bloqueo pesimista.

"¿Qué es el bloqueo pesimista y por qué lo usás?" @Lock(PESSIMISTIC_WRITE) genera un SELECT ... FOR UPDATE : bloquea la fila hasta terminar la transacción. Lo uso en el login para que dos requests simultáneos no pisen el contador de intentos fallidos y la cuenta se bloquee correctamente.

"¿Por qué hay repositories vacíos?" Porque igual necesito el objeto para llamar a los métodos heredados de JpaRepository : save , findById , findAll . La interfaz vacía es suficiente.

"¿Por qué usás countBy en vez de traer la lista y contarla?" Porque un COUNT(*) en la base es mucho más barato que traer todas las filas y contarlas en Java. En el contador de notificaciones no leídas, que se consulta cada 15 segundos, la diferencia es importante.

Índice de repositories cubiertos en este documento

Los 26 están listados en la tabla resumen de arriba, con su entidad, el tipo de su id y cuántos métodos custom tiene cada uno.

Los tres a destacar si tenés que elegir:

- UsuarioRepository — los únicos dos @Query con @Lock del proyecto, y el tema de concurrencia.

